



Безперервна інтеграція, доставка та розгортання CI/CD



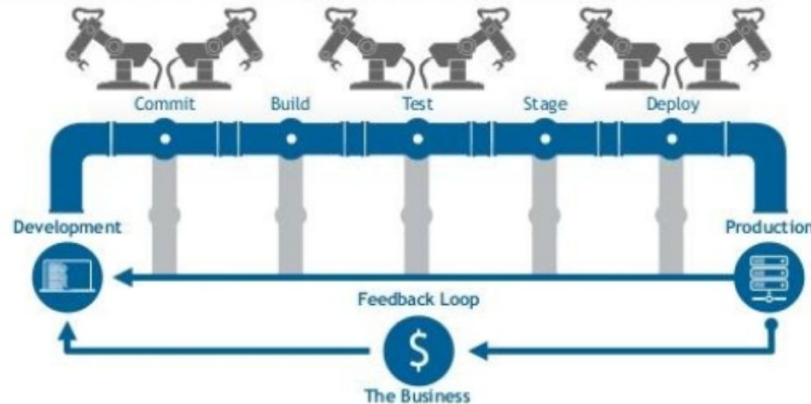
Порядок денний

- Вступ і визначення
- Як це працює?
- Переваги використання CI/CD
- Принципи
- Інструменти та приклади
- Майбутні тенденції в CI/CD
- Питання та відповіді

Антишаблони доставки програмного забезпечення

- Розгорніть програмне забезпечення вручну
- Розгортання в середовищі, схожому на виробниче (наприклад, програвання), лише після завершення розробки
- Управління конфігурацією для виробничого середовища вручну

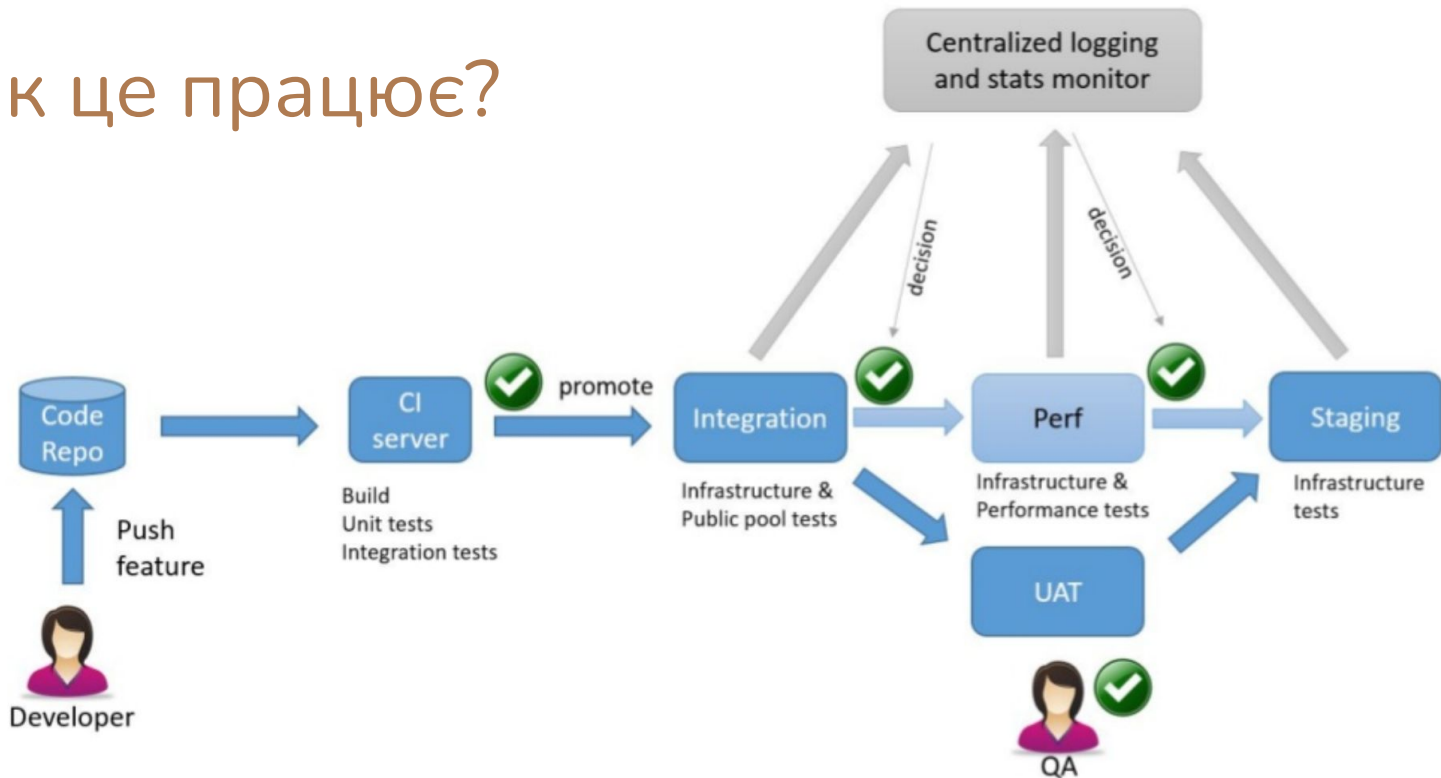
Continuous Delivery and Automation are Key



Визначення

- *Безперервна інтеграція* зазвичай відноситься до інтеграції, створення та тестування коду в середовищі розробки. *Безперервна доставка* спирається на це, займаючись завершальними етапами, необхідними для розгортання виробництва
- *Безперервна доставка* — це підхід до розробки програмного забезпечення, за якого команди створюють програмне забезпечення за короткі цикли, гарантуючи, що програмне забезпечення може бути надійно випущено в будь-який час. Вона спрямована на створення, тестування та випуск програмного забезпечення швидше та частіше – Wiki
- *Безперервне розгортання* означає, що кожна зміна проходить конвеєр і автоматично запускається у виробництво, що призводить до багатьох виробничих розгортань щодня. Щоб виконувати *безперервне розгортання*, ви повинні виконувати *безперервну доставку*

Як це працює?



Переваги CI/CD

- Зменшення повторюваних ручних процесів → автоматизуйте все
- Зменшення помилок (наприклад, помилок людини)
- Зниження стресу: швидке та просте розгортання/відкат
- Помилки та проблеми, виявлені ближче до моменту їх появи, покращують продуктивність і якість
- Систему можна розгорнути частіше та гнучкіше, що забезпечує більше відгуків користувачів → надійні випуски
- Підвищення рівня задоволеності клієнтів

Виклики без CI/CD

- **Ручні процеси:** Схильні до помилок і потребують багато часу.
- **Непоследовні середовища:** Відмінності призводять до помилок і проблем із розгортанням.
- **Проблеми інтеграції:** Складно об'єднувати зміни, що викликає конфлікти.
- **Повільний зворотний зв'язок:** Затримки у виявленні та виправленні проблем.
- **Низька частота випусків:** Рідкі оновлення обмежують оперативність.
- **Забезпечення якості:** Ручне тестування може пропускати дефекти.
- **Проблеми масштабованості:** Важко підтримувати послідовність у великих командах і проєктах.

CI/CD у контексті гнучкої (Agile) розробки

- **CI/CD автоматизує процеси інтеграції та розгортання**, що тісно узгоджується з принципами Agile — ітеративною розробкою, безперервним удосконаленням і швидким зворотним зв'язком.
- **Часті інтеграції та автоматизовані розгортання** забезпечують швидке впровадження нових функцій і виправлень, підсилюючи цикли Agile.
- **Автоматизоване тестування** у конвеєрах CI/CD надає миттєвий зворотний зв'язок після кожної зміни, що є критично важливим для адаптивного планування та еволюційної розробки в Agile.
- **CI/CD сприяє кращій співпраці** між розробниками, тестувальниками та операційними командами, підтримуючи єдине джерело істини й автоматизуючи потік змін.
- **CI/CD масштабує фреймворк Agile** на більші та складніші проєкти завдяки стандартизації процесів розробки, тестування та розгортання.

Параметри інструменту CI/CD



Jenkins



Bamboo

go
Continuous Delivery



Travis CI



circleci



GitLab



CODESHIP



powered by

amazon
web services™

Visual Studio Team Services



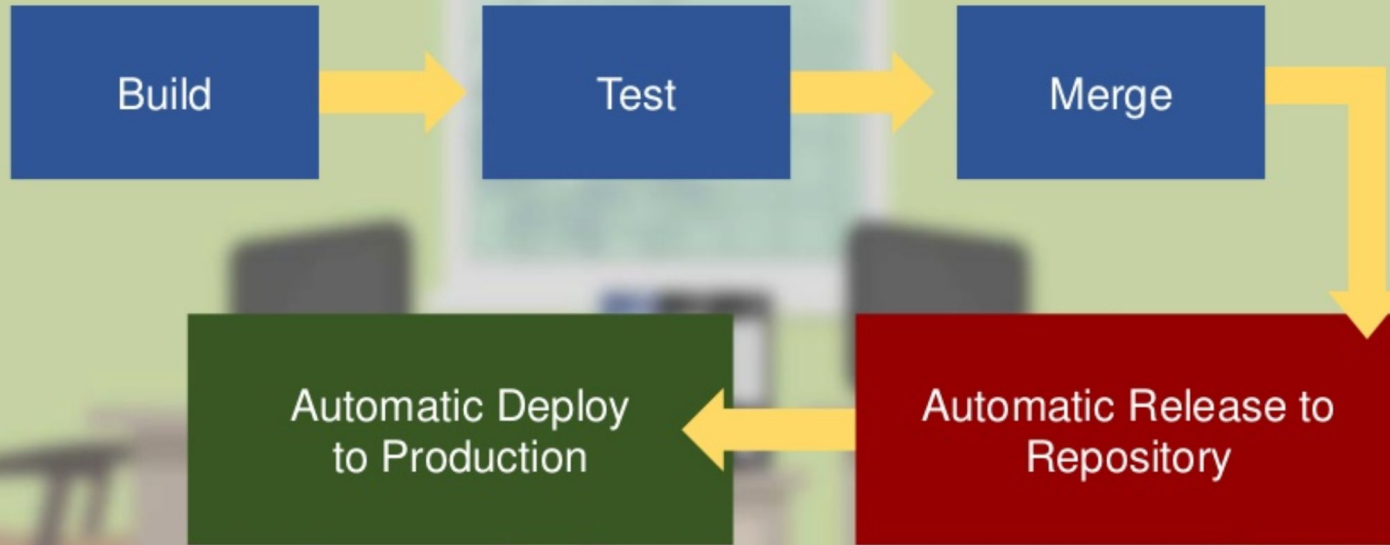
Приклад CI/CD – Verizon Prepaid

- **Огляд проекту:** Verizon — це веб-програма, яка розміщує AWS для підтримки інтерфейсу для обробки транзакцій кінцевих клієнтів через робочий процес для активації облікових записів Verizon Prepaid
 - Технологічний стек: Node.js, Angular.js, Docker, AWS
 - Команда: 4 розробники, 1 розробник автоматизації, 1 QA
- **Процес розробки**
 - Дві тривалі гілки освоюють і розвивають із перехідною кількістю функціональних гілок
 - Використання GIT Flow
 - робочий процес
 - Створити гілку функції з розробки
 - Внести зміни до гілки функції
 - Створити запит на вилучення (PR), націлений на розробку
 - Об'єднаний PR після того, як він пройшов тести CI та перевірку команди
 - Видалити гілку функції після завершення інтеграції



Принципи

CONTINUOUS INTEGRATION

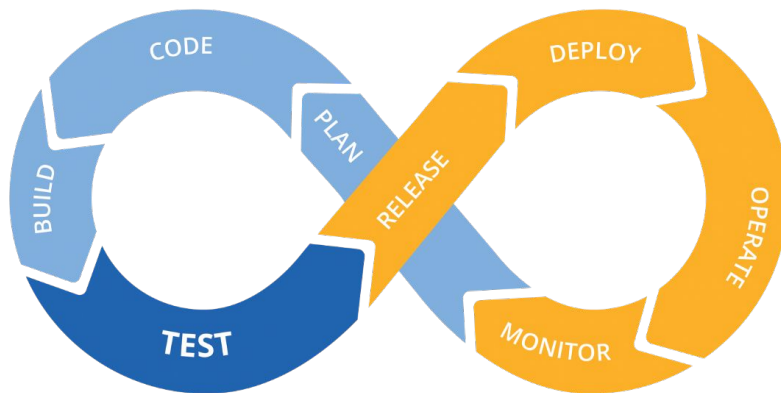


CONTINUOUS
DEPLOYMENT

CONTINUOUS
DELIVERY

Що таке безперервна інтеграція?

Безперервна інтеграція (CI) — це практика розробки програмного забезпечення DevOps, яка дозволяє розробникам об'єднувати зміни свого коду в центральній репозиторії, щоб можна було запускати автоматизовані збірки та тестування.



Що таке безперервна інтеграція?

Безперервна доставка зараз цінується організаціями будь-якого розміру, і вони готові інвестувати в неї з таких причин:

1

Уникає
конфліктів
злиття

2

Зменшує
перегляд
коду

3

Прискорює
процеси
розвитку

4

Зменшує
відставання
проекту

Що таке безперервна доставка?

Continuous Delivery (CD) — це практика DevOps, яка стосується створення, тестування та впровадження вдосконалень програмного коду. Найважливішою частиною CD є те, що код завжди знаходиться в стані, де можна розгортати



Що таке безперервна доставка?

Безперервна доставка зараз цінується організаціями будь-якого розміру, і вони готові інвестувати в неї з таких причин:

1

**Продукти
вищої якості**

2

**Швидші та
менш
ризиковані
звільнення**

3

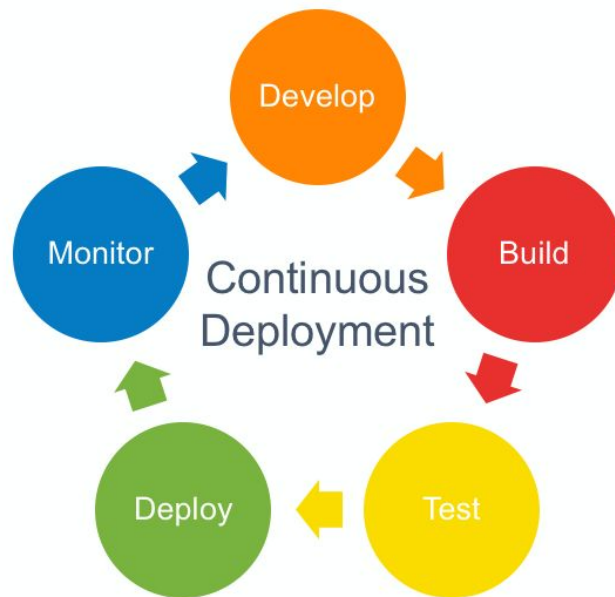
**Клієнти
більш
задоволені**

4

**Більше
ефективності та
економічності
команди**

Що таке безперервне розгортання?

Безперервне розгортання (CD) відноситься до останнього етапу в конвеєрі, який відноситься до автоматичного випуску будь-яких змін розробника зі сховища в виробництво.



Що таке безперервне розгортання?

Безперервне розгортання зараз цінується організаціями будь-якого розміру, і вони готові інвестувати в нього з таких причин:

1

**Швидший
розвиток
продуктів**

2

**Менш ризиковані
випуски та
проблеми, які
легко виправити**

3

**Постійне
вдосконален
ня якості**

Різниця між Continuous Delivery та Continuous Deployment

Безперервна доставка (Continuous Delivery):

- **Мета:** автоматизувати процес випуску програмного забезпечення, щоб будь-яку версію можна було розгорнути у продакшн у будь-який момент.
- **Процес:** зміни в коді автоматично збираються, тестуються та готуються до випуску, але саме розгортання зазвичай виконується вручну.
- **Фокус:** забезпечити готовність кожної зміни до випуску, підвищуючи впевненість у якості нових релізів.

Безперервне розгортання (Continuous Deployment):

- **Мета:** автоматизувати процес розгортання, щоб мінімізувати час між написанням коду та його використанням у продакшн-середовищі.
- **Процес:** розширює безперервну доставку, автоматично розгортаючи всі зміни, що пройшли тестування, без ручного втручання.
- **Фокус:** прискорити загальний процес випуску, забезпечуючи миттєвий зворотний зв'язок від користувачів і безперервне вдосконалення.

Стратегії безпечного розгортання

- **Canary Releases (Канарейкові релізи):** поступове впровадження змін для невеликої групи користувачів, щоб відстежити продуктивність і виявити проблеми до повного розгортання.
- **Blue/Green Deployments (Синьо-зелене розгортання):** нова версія застосунку розгортається поруч зі старою. Трафік можна переключати поступово або миттєво, залежно від стабільності нової версії.
- **A/B Testing Deployment:** одночасне тестування двох версій застосунку для порівняння їхньої продуктивності, зручності використання та інших метрик перед масштабним релізом.
- **Ramped Deployment (Поступове оновлення):** поступова заміна екземплярів старої версії застосунку новими на різних серверах, що знижує ризики завдяки розподілу оновлення в часі.
- **Shadow Deployment (Тіньове розгортання):** нова версія розгортається поряд зі старою у реальному середовищі, але до неї спрямовується лише внутрішній трафік. Це дозволяє протестувати систему без впливу на кінцевих користувачів.
- **Recreate Deployment (Пересоздання розгортання):** стара версія застосунку тимчасово зупиняється, а потім запускається нова. Простий підхід, але з коротким періодом недоступності (downtime).

Моніторинг і дії після розгортання

- **Моніторинг у реальному часі:** використовуйте інструменти на кшталт *Prometheus* і *Grafana* для безперервного відстеження продуктивності системи.
- **Сповіщення та нотифікації:** автоматизуйте сповіщення через платформи, такі як *Slack* або електронна пошта, щоб миттєво отримувати інформацію про проблеми.
- **Керування журналами (Log Management):** застосовуйте *ELK Stack* або *Splunk* для ефективного аналізу логів і діагностики несправностей.
- **Метрики продуктивності:** використовуйте інструменти *APM* (Application Performance Monitoring) для контролю ключових показників, таких як час завантаження, затримки та рівень помилок.
- **Безперервний цикл зворотного зв'язку:** створіть механізми зворотного зв'язку для постійного вдосконалення системи на основі користувацьких даних і показників продуктивності.

Як краще організувати CI/CD?

- Безперервна інтеграція та безперервна доставка є найкращими методами, оскільки вони створюють ефективний процес інтеграції та доставки продукту на ринок
- У програмний код можна внести невеликі зміни, що спростить і полегшить весь процес
- CI та CD забезпечують постійний зворотний зв'язок від клієнтів, а також від команди DevOps. Це підвищує прозорість будь-якої проблеми в команді чи за її межами
- Процес дозволяє швидше випускати продукт. Тепер несправності можна швидше виявляти й, отже, швидше виправляти, що збільшує швидкість звільнення

Безпека в CI/CD

- Інтегруйте **автоматизовані перевірки безпеки** за допомогою інструментів, таких як *SonarQube* та *OWASP ZAP*, щоб аналізувати код на наявність вразливостей.
- **Керуйте секретами безпечно** за допомогою *Vault by HashiCorp* або *AWS Secrets Manager*, щоб уникнути їхнього розголошення у кодовій базі.
- **Запровадьте практики рецензування коду** та автоматизовані перевірки для дотримання стандартів безпечного програмування.
- Використовуйте **framework-и типу “policy-as-code”**, наприклад *Open Policy Agent*, щоб автоматично забезпечувати дотримання політик безпеки.
- Реалізуйте **безперервний моніторинг безпеки**, щоб виявляти та реагувати на загрози в режимі реального часу.

Масштабованість та ефективність

- **Контейнеризація та оркестрація:** Використовуйте *Docker* для контейнеризації, щоб ізолювати застосунки у портативних середовищах. Застосовуйте *Kubernetes* для автоматичного масштабування контейнерів залежно від навантаження та трафіку, оптимізуючи використання ресурсів.
- **Архітектура мікросервісів:** Розділіть застосунок на менші, незалежні сервіси, які можна масштабувати окремо. Такий підхід дозволяє здійснювати цільове масштабування й оновлення, зменшуючи простоту та покращуючи швидкість відгуку.
- **Оптимізація конвеєра (pipeline):** Виконуйте завдання в CI/CD-конвеєрі паралельно, щоб скоротити час збірки та розгортання. Реалізуйте стратегії кешування для повторного використання раніше отриманих або скомпільованих даних, що значно прискорює процес.
- **Моніторинг продуктивності:** Регулярно відстежуйте такі показники, як використання CPU, споживання пам'яті та час відгуку. Використовуйте ці дані для динамічного коригування ресурсів, забезпечуючи ефективну роботу без надлишкового виділення.
- **Інфраструктура як код (IaC):** Застосовуйте інструменти на кшталт *Terraform* для керування розгортанням та підготовкою інфраструктури за допомогою коду. Це дає змогу швидко відтворювати перевірені середовища, зменшує кількість людських помилок і підвищує узгодженість розгортань.



Майбутні тенденції в CI/CD

Штучний інтелект та машинне навчання в CI/CD

- Використовуйте моделі машинного навчання для прогнозування та запобігання збоїв під час розгортання шляхом аналізу історичних даних.
- Інтегруйте AI для автоматизації та оптимізації створення та аналізу тестових сценаріїв, підвищуючи ефективність тестування та покриття.
- Покращуйте системи моніторингу за допомогою AI для автоматичного виявлення аномалій, що дозволяє швидше реагувати на проблеми продуктивності.
- Застосовуйте алгоритми машинного навчання для оптимізації розподілу та використання ресурсів, забезпечуючи ефективну роботу з мінімальними витратами.
- Реалізуйте механізми безперервного навчання, щоб AI-системи могли еволюціонувати та адаптувати процеси на основі зворотного зв'язку з операцій.

Інфраструктура як код

Інфраструктура як код (Infrastructure as Code, IaC) використовує код для керування та розгортання інфраструктури, автоматизуючи налаштування серверів, баз даних та інших мережевих компонентів. Такий підхід забезпечує узгодженість і швидкість під час створення та масштабування середовищ. Інструменти, такі як **Terraform**, **Ansible** та **AWS CloudFormation**, дозволяють розробникам визначати вимоги до інфраструктури за допомогою коду, який можна зберігати у системах контролю версій і повторно використовувати, мінімізуючи людські помилки та спрощуючи процеси розгортання. IaC підтримує швидке ітераційне оновлення та відтворюваність у середовищах розробки, тестування та продуктивного використання, забезпечуючи ефективне керування й оперативні зміни відповідно до потреб.

Роль хмарних сервісів

Хмарні сервіси відіграють ключову роль у конвеєрах CI/CD, надаючи масштабовані та гнучкі ресурси, які можна динамічно налаштовувати відповідно до потреб процесів розробки та розгортання. Вони пропонують основні сервіси, такі як хмарні сервери для збірки, сховища даних і середовища, що сприяють швидшому та ефективнішому виконанню конвеєрів. Платформи на кшталт **AWS**, **Azure** та **Google Cloud** надають інструменти, які безпосередньо інтегруються у робочі процеси CI/CD, підтримуючи такі дії, як автоматизоване тестування, розгортання та моніторинг. Хмарні сервіси дозволяють організаціям використовувати передові технології без необхідності значних початкових витрат на фізичну інфраструктуру, зменшуючи накладні витрати та підвищуючи гнучкість.

Інтеграція DevSecOps

DevSecOps інтегрує практики безпеки у конвеєр CI/CD, вбудовуючи безпеку на кожному етапі розробки та експлуатації програмного забезпечення. Такий підхід гарантує, що питання безпеки розглядаються не після завершення розробки, а є невід'ємною складовою всього процесу. Основні практики включають **автоматизоване сканування безпеки, безперервний моніторинг відповідності стандартам** і використання підходу «**безпека як код**» (**Security as Code**) для програмного забезпечення політик безпеки. DevSecOps формує культуру тісної співпраці між командами розробки, експлуатації та безпеки, що сприяє швидшому виявленню й усуненню вразливостей, підвищуючи загальний рівень безпеки застосунків.

Питання та відповіді